Embedded Operating Systems and Architectures notes

Luca Seggiani
2026

1 Lesson 23-09-26

1.1 Introduction

Computing systems are everywhere: cars, trains, aircraft, satellites, medical devices, printers, phones, industrial controllers. In an **embedded** system, we consider *software plus hardware*, not software alone: the program only makes sense together with the machine it drives. Complexity is growing dramatically. A modern car carries ≈ 100 electronic control units and on the order of 10^8 lines of on-board code, against a handful of hundreds of lines in the 1970s.

1.1.1 Operating systems

The natural question to ask is *why* an embedded system might need an operating system. The answers are many:

- Bare metal applications no longer cope with the complexity even in the embedded systems domain;
- **An operating system raises the abstraction level:** it gives application code a small set of rules and services instead of ad-hoc hardware handling;

Our goal is to connect three things that are usually taught apart: **computer architecture**, **operating systems**, and the **computer system** that has to run both.

1.1.2 Topics

The main roadmap of the course is:

- We will connect operating-system concepts to executable embedded software;
- This course lives in the **platform** layer of a computing node: from the hardware up to the system-call interface, with the application used as the thing that exercises it.

The covered topics are:

- **Minimal computing-system architecture** — juste enough to write operating systems code;
- **Bare-metal development flow** — cross-compilation, emulators and simulators, debugging and profiling, virtual platforms;
- **Operating systems for embedded systems** — real-time and process scheduling, the anatomy of an embedded OS, architecture and services of real systems (FreeRTOS);
- **Programming an embedded OS** — building it, device management and drivers, drivers for custom devices.

1.2 Introduction to embedded operating systems

An **OS** (*Operating System*) is nothing but a program, with a very simple purpose: to mediate between hardware and the programs that use it.

There are three classic ways to draw that boundary:

- **Monolithic**: kernel is one large program; drivers, filesystems, scheduling all share one address space. Both Linux and Windows fit this description.

  Monolithic kernels are simple to implement once the task switching machinery is running: once we can switch to kernel mode, we can touch any resource available to the system to carry out tasks;
- **Microkernel**: kernel does the bare minimum (IPC, scheduling, basic memory); everything else runs as separate services.

  Microkernels are interesting because they are inherently safer: the service handling the filesystem, for instance, cannot corrupt other services on failure (as it runs itself as a protected service with no access to other resources). The problem comes up when complex tasks have to performed: the main overhead is given by the IPC and scheduling facilities which sit inbetween services;
- **Flat**: no kernel/user separation at all; a small set of statically-defined tasks share the CPU under one scheduler.

  This is important to us because embedded systems often don't feature protection systems, and as we know complete protection is impossible without hardware support.

The core of the operating system is basically the *scheduler*: everything else can be seen as a task either the operating system or the user programs carry out, based on the resources allocated by the scheduler.

1.2.1 Hardware resources

The purpose of an OS is to make hardware *usable*. Without an OS, every application must decide how to initialise the CPU, program a timer, share a UART, and recover from an interrupt. An OS supplies a small set of rules and mechanisms so application code can ask for useful services instead of repeatedly rebuilding hardware policy.

The resources are finite, especially on a small board:

- **CPU time**: which ready task runs now? We also haveto take into account the fact that CPUs interact with many other devices on the same board;
- **memory**: which code and data occupy RAM and Flash?
- **devices**: who may configure a timer or consume a received byte?
- **events**: how does an interrupt become work for the right task?

1.2.2 Types of operating systems

The three labels we gave earlier describe where operating-system services execute and how strongly they are isolated.

Type	Services offered	Tradeoff
Monolithic	Together in kernel space	Direct calls, large trusted core
Microkernel	Mostly as separate servers	Isolation, IPC overhead/design
Flat	Together in one firmware image	Tiny and direct, little isolation

Monolithic A monolithic kernel puts scheduling, drivers, filesystems, and networking in one kernel address space. Services can call each other directly and quickly.

- Linux is the familiar large-scale example
- A faulty driver can corrupt kernel state

- The design is practical when the feature set is broad and tightly coupled

On a protected machine, applications normally execute outside this trusted kernel address space.

Microkernel A microkernel retains only the most fundamental mechanisms in the kernel, usually scheduling, address-space management, and inter-process communication.

Drivers and other services become separate programs that exchange messages.

- Smaller trusted computing base
- A service failure can be contained or restarted
- More interfaces and message paths to design correctly

seL4 and QNX are useful reference points for this family.

Flat Flat embedded firmware usually has one linked image and one physical address space. Tasks cooperate through shared memory and the scheduler.

There is no process loader, no virtual-memory translation, and usually no hardware-enforced boundary between tasks. This puts more pressure on software developers, as user code must be free of bugs which could potentially crash the whole operating system. This is particularly critical in systems like vehicles, medical or industrial. We know that monolithic operating systems give us an extra layer of safety, by switching to a safer kernel mode to perform critical tasks, which user code can't normally access.

1.2.3 Operating system roadmap

Where does this course sit in this hierachy?

- **Linux** (a monolithic kernel) and *seL4/QNX* (microkernels) are the extremes we'll read about;
- This course builds **flat** first (`scratch-os`) — the simplest point on the spectrum, and the one that makes the boot-to-scheduling pipeline fully visible. `scratch-os` is intentionally small. Its value is that every important step can fit in mind;
- Then **FreeRTOS** — closer to a minimal microkernel-flavored RTOS: a small kernel, real scheduling policies, but still no memory protection between tasks by default.

  FreeRTOS provides established implementations of the mechanisms we first meet in `scratch-os`:

  – Task creation and priorities;
  – Delay and timer-driven scheduling;
  – Queues, semaphores, and mutexes;
  – Interrupt-to-task communication.

  The comparison matters: an API call becomes easier to reason about when its underlying state transitions are no longer invisible.

1.2.4 Real-time operating systems

The **real-time** label applied to an OS is about *bounded response*, not raw speed. The idea is that we can know exactly how long a task will take before executing it. A result computed after its deadline can be a failure even when its value is otherwise correct. This is particularly useful in embedded systems which might have external (e.g. physical) constranints, in addition to technical ones.

Requirement	Example
Functional	sample a sensor correctly
Temporal	sample it every 1 ms, before the deadline
Robustness	continue safely when one input is missing

1.2.5 Embedded operating systems

Embedded operating systems usually look different than traditional ones:

- **Resource-constrained**: kilobytes of RAM/flash, not gigabytes — no room for a general-purpose kernel's overhead;
- **Real-time**: correctness often depends on *when* something happens, not just *whether* it happens;
- **No user protection needed (usually)**: one vendor, one firmware image, one set of tasks — the isolation problems a desktop OS solves mostly don't apply here.

  This is mainly a cost problem: it's not that car manufactures don't *want* safety in their devices, but that that CPUs which don't feature protection mechanisms are usually cheaper;
- These constraints are why `scratch-os` can be simple and still be a real, working OS.

An embedded OS is selected and configured around a particular product, not an unknown future workload.

- Static task counts and fixed buffers can replace dynamic allocation;
- A simple priority scheduler can replace a general-purpose fairness policy;
- Direct register access can replace a layered device stack;
- Predictable execution is often more valuable than maximum throughput.

Small is not incomplete when it matches the system's responsibilities.

Many **MCU** (*Micro Controller Unit*) products ship one firmware image that includes startup code, drivers, the OS, and application logic. Updating that image changes the whole system.

This simplifies deployment, but it raises the bar for discipline:

- a bad pointer can damage any writable state
- a blocked high-priority task can stop an important service
- recovery and observability must be designed into the firmware

Later, we will see how scheduling and synchronisation reduce these risks.

1.2.6 Embedded systems

There are many ways we can classify **embedded systems**:

- A computer built into a larger device to do **one dedicated job** — not a general-purpose machine you install arbitrary software on;
- Hardware + software (often called *firmware*) shipped as one unit; the end user never sees a shell or a package manager;
- Everywhere: engine controllers, pacemakers, drones, washing machines, Wi-Fi routers, the microcontroller in a USB charger;
- The population of embedded processors passed the human population decades ago — most CPUs made are never in a "computer".

We can compare embedded systems to general-purpose ones (such as modern computer or phone operating systems):

	Embedded	**General-purpose**
Workload	one fixed task	anything the user installs
Resources	KB of RAM/flash	GB of RAM, TB of disk
Timing	often hard deadlines	best-effort
User model	no interactive user	multi-user, multi-tasking
Cost/power	pennies, milliwatts	dollars, tens of watts

The right-hand column is what a desktop OS is built for; `scratch-os` targets the left-hand one, which is *why* it can be small.

1.3 Microcontrollers

We begin to approach the device that we'll actually need to program. A **microcontroller** or **MCU** (*Micro Controller Unit*) integrates onto a single chip what a desktop splits across a whole motherboard:

- **CPU core** — executes instructions (ours: ARM Cortex-M3);
- **RAM** — volatile working memory, tens of KB;
- **Flash** — non-volatile program storage; code runs directly from it;
- **Peripherals** — timers, UART, GPIO, ADC, all on-chip;
- One chip, one clock, one address space — no external memory bus to wire up, no BIOS, no disk.

Compared to desktop or handled general-purpose CPUs, we note that:

- A desktop CPU is *just* the core: RAM, storage, and I/O are separate chips reached over buses; an MMU gives every process its own virtual address space;
- An MCU has **one flat physical address map** shared by code, data, and every peripheral register — no MMU, no virtual memory (a Cortex-M has at most an optional MPU);
- On an MCU, "the hardware" is *memory you write to*: there is no driver layer between your code and a device register.

1.3.1 Reset routine

As an example, let's look into what happenes to a Cortex-M CPU at reset time. After reset, the Cortex-M does not discover an operating system. It reads two words from the vector table in Flash:

1. The initial value for the main stack pointer
2. The address of `Reset_Handler`.

The reset handler is our first code. It establishes the C runtime environment before calling `main()`.

1.3.2 Registers

The CPU operates on values held in registers. Memory is accessed explicitly with load and store instructions.

Register role	Meaning
Program Counter	Selects the next instruction
Stack Pointer	Locates calls, local variables, saved state
General Registers / Programmer Registers	Hold operands and addresses
Status Register	Records conditions and interrupt state

When an exception occurs, the processor automatically saves essential state on the current stack so execution can later resume.

1.3.3 Memory

We can talk about the *memory* available to an MCU. Usually there are two kinds of memory:

- **SRAM / DRAM**: not persistent, but extremely fast. Used as main program memory;
- **Flash** memory: read-only, but still very fast and fit (in embedded systems) for containing code. This is in accord with the usual definition of *Harvard Architecture*. We should note that most general purpose devices use the opposite architecture, that is *Von Neumann Architecture* (which use program memory both for code and data).

Flash keeps its contents without power. SRAM is fast writable working memory whose contents disappear on reset or power loss.

```
Flash: vector table | .text | .rodata   | initial bytes for .data
SRAM:  .data        | .bss  | heap area | stack
```

This split explains why startup code must copy some bytes and clear others before ordinary C code can safely run.

2 Lesson 30-09-26

2.1 Memory

Last week we went over the structure of an embedded system MCU. Now we focus on memory, with the target of defining a memory map and defining a linker script for it.

2.1.1 Types of memory

Let's quickly go over the *types* of memory available to us.

SRAM

- **SRAM** (static RAM) sits on the CPU's memory bus: every byte has an address, and the CPU reads or writes it with a normal load/store;
- **Fast**: on a microcontroller, one access takes about one clock cycle;
- **Byte-writable**: we can change a single byte, any time, as often as we like;
- **Volatile**: power off and everything is lost; at power-on the content is **undefined** (random bits, not zeros).

SRAM is where anything that **changes** while the program runs must live: variables, the stack.

NOR flash

- Also sits on the memory bus, like a ROM: every word has an address, random-access reads are fast;
- So the CPU can **fetch** instructions directly from it (common in MCUs): **execute-in-place** (XIP). No copy needed;
- **Non-volatile**: the program is still there after power-off;
- Writing is **not** a store instruction:
  - First **erase** a whole sector (e.g. a few KB) $\rightarrow$ all bits become 1;
  - Then **program** words (bits can only go $1 \rightarrow 0$)
  - Microseconds to milliseconds, through a special command sequence
- Result: at run time, NOR flash is effectively **read-only**.

NAND flash and disks

- NAND flash (SD cards, SSDs) and hard disks are **not** on the memory bus: they are reached through a controller, one **block** (page) at a time, e.g. 512 B – 4 KB;
- A block has no CPU address $\rightarrow$ the CPU **cannot fetch instructions** from it, nor read a single variable;
- Contents must first be **copied into RAM**, then used from there;

This is what a desktop PC does: the program sits on the SSD, and the OS **loader** copies it into RAM before running it. A microcontroller has no loader: its program already sits in NOR flash, at the address it runs from.

2.1.2 Effect on program sections

The different memory types we have available play a role in how we define our *program headers* (or *segment* headers) for embedded programs.

Technology	On the bus	Executable	Writable at runtime	Persistent
SRAM	Yes	Yes (XIP)	Yes, bytes	No
NOR flash	Yes	Yes (XIP)	No (erase/program)	Yes
NAND / disk	No (blocks)	No	Via a driver	Yes

The CPU sees one 4 GB address space. On every Cortex-M the **regions** are fixed by the architecture; each chip decides what it populates.

Address range	Region	On `mps2-an385`
`0x0000_0000–0x1FFF_FFFF`	Code	4 MB at `0x0`: the program ("FLASH")
`0x2000_0000–0x3FFF_FFFF`	SRAM	4 MB at `0x2000_0000`: "RAM"
`0x4000_0000–0x5FFF_FFFF`	Peripherals	UART, timers…
`0x6000_0000–0xDFFF_FFFF`	External RAM / devices	Not used here
`0xE000_0000–0xE00F_FFFF`	CPU's private registers	Control of the CPU itself (at `0xE000_E000`)

Tabella 1: Cortex-M memory map and the regions populated by `mps2-an385`.

2.1.3 Linker script

This leaves us with the following linker script, which we can find very useful to analyze:

```
/* Linker script for mps2-an385 machine (ARM Cortex-M3, MPS2 platform). */

/* Physical memory regions of the board. */
MEMORY
{
    /* code, rodata (XIP) */
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M

    /* data, bss, stack */
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

/* Entry point, also entry 1 of vector table (reset handler). */
ENTRY(Reset_Handler)

/* initial SP = top of RAM; the stack grows downward from here */
_estack = ORIGIN(RAM) + LENGTH(RAM);

/* 4 KB reserved for the stack*/
_Min_Stack_Size = 0x1000;

/* combine input (ELF) sections into output sections */
SECTIONS
{
    /* Vector table first, at address 0, so the CPU finds the initial SP
     * and reset vector where the architecture requires them. */
    .isr_vector :
    {
        . = ALIGN(4);            /* vector table must be word-aligned */
        KEEP(*(.isr_vector))     /* KEEP: never garbage-collect it  */
        . = ALIGN(4);            /* keep alignment */
    } > FLASH                    /* place in FLASH */

    /* Executable code and read-only data. */
    .text :
    {
        . = ALIGN(4);           /* word-align the start of this section */
        *(.text*)               /* all code from every input object */
        *(.rodata*)             /* string literals, const tables */
        . = ALIGN(4);           /* word-align the end so (_etext) */
        _etext = .;             /* end-of-text marker */
    } > FLASH                   /* place in FLASH */

    /* FLASH address where .data's initial image is stored: the technique
    allows for reserving space in RAM while storing in flash. */
    _sidata = LOADADDR(.data);

    /* Initialized variables: they live (run) in RAM but their initial
     * contents are stored in FLASH and copied over by Reset_Handler. */
    .data :
    {
        . = ALIGN(4);           /* word-align the start of .data */
        _sdata = .;             /* start of .data in RAM */
        *(.data*)               /* every initialized-variable input
    section */
        . = ALIGN(4);           /* word-align the end */
        _edata = .;             /* end of .data in RAM - copy loop stops
    here */
    } > RAM AT > FLASH          /* place in FLASH, access in RAM */

```

```
59      /* Zero-initialized variables: reserve RAM, no stored image. */
60      .bss :
61      {
62          . = ALIGN(4);           /* word-align the start of .bss */
63          _sbss = .;              /* start of .bss */
64          *(.bss*)                /* every zero-initialized input section */
65          *(COMMON)               /* legacy tentative-definition symbols */
66          . = ALIGN(4);           /* word-align the end */
67          _ebss = .;              /* end of .bss - zero loop stops here */
68      } > RAM
69  }
```

For now we intentionally skip the *heap* and other advanced features, but this should be pretty useful to understand how the memory is laid out. Let us remember that here we are defining memory **regions**, declaring memory **sections**, and expecting the linker to generate for us memory **segments** (program headers) which place the data in our segments within more "blocks" (that is, segments) to load in different memory regions (the ones seen above).

2.1.4 LMA vs VMA

Let us also note the *LMA/VMA* distinction:

- **LMA** (*Load Memory Address*) is where the initial bytes of data are located in the executable/image (the program headers which we load);
- **VMA** (*Virtual/Run-time Memory Address*) is where the data in the section expects to live when the program executes.

The gimmick at lines 45-56 allows us to set a LMA for the data section inside the flash memory, and set its VMA to the address it will be copied to in RAM. Of course, the task of copying the data itself is assigned to the reset handler.

We also note that the `.bss` initialize sector has a VMA (where to access uninitialized data) but no LMA (nothing to load, the reset handler just zeroes out).

2.1.5 Vector table

One important region of the memory (more specifically, FLASH memory) is the **vector table**.

- It is a **table** of function addresses at a fixed place (address `0x0` at reset).
- Each **exception** (an event that makes the CPU stop and run a specific handler: reset, a fault, ...) has a numbered **slot**; the CPU reads slot N to find where to jump,
- Slot 0 is special: it contains not an address of code, but the initial stack pointer (SP), which gets loaded automatically by the CPU. This means that the reset handler is at index 1 (not 0).

In `boot/startup.s` (base), section `.isr_vector`:

```
1  isr_vector:
2      .word _estack           /* 0: initial SP  (C: &_estack)      */
3      .word Reset_Handler     /* 1: reset       (C: Reset_Handler) */
```

The stored value of word 1 is `Reset_Handler | 1`: the low bit marks Thumb code. Nothing ever *calls* `isr_vector`; only the hardware reads it.

2.2 Registers

We now go over the ARM architecture programmer registers. In all ARM processors, the following registers are available and accessible in any processor mode:

- 13 general-purpose registers: `R0-R12`: user variables, with call arguments passed through `R0-R3`;
- One *Stack Pointer* (`SP`): address of the top of the stack;
- One *Link Register* (`LR`): where to return after a function call;
- One *Program Counter* (`PC`): address of the next instruction to execute;
- One *Status Register* (`xPSR`): flags, Thumb bit, current exception.

Some things to note at this level which may not be clear:

- There are no "variables" at this level: only these registers and memory.
- The stack grows **downwards**: a push first does $\mathrm{SP} -= 4$, then writes.

xPSR packs three things into one register:

Bits	Content
31–28	flags **N Z C V** (negative, zero, carry, overflow), set by `cmp`
24	**T**: Thumb state — must always be 1 on Cortex-M
8–0	number of the exception being handled (0 = none)

Tabella 2: xPSR fields

The CPU is always in one of two **modes**:

- **Thread mode** — normal code: `Reset_Handler`, `main()` (xPSR bits 8–0 = 0).
- **Handler mode** — running an exception handler (bits 8–0 = its number).

Code can also be *privileged* or not; everything in this unit runs privileged.

2.3 ISA

ARM Cortex-M uses a **load/store** architecture: arithmetic and logical operations operate on registers, while memory is accessed explicitly through load and store instructions.

- `ldr`: **load** data from memory into a register;
- `str`: **store** data from a register into memory;
- Arithmetic instructions do not directly operate on memory operands.

Let's have a table of example memory operations:

Assembly	C equivalent
`ldr r3, [r0]`	`r3 = *r0;`
`str r3, [r1]`	`*r1 = r3;`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;`
	`// i.e. r3 = *src++;`
`str r3, [r1], #4`	`*r1 = r3; r1 += 4;`
	`// i.e. *dst++ = r3;`
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;`
	`// one byte, at offset 7`

2.4 Post- and pre-increment

In ARM we have **pre-indexed** and **post-indexed** memory operations, that is control over the time when the base register is updated.

Being more specific, ARM allows load and store instructions to optionally *alter* the address contained within a base register. 3 forms are available:

- `ldr r0, [r1, #4]` is equivalent to immediate offset addressing in RISC-V, with `#4` being the immediate operand;
- `ldr r0, [r1, #4]!`, with the additional `!`, means performing a *write-back* or **pre-indexing** operation. The calculated address, obtained by adding the offset, is *written back* into the base register before the load operation. The result we get is that of a load from the base address plus the offset, with the base address reflecting the change after execution. Compared to RISC-V, this saves an instruction;
- `ldr r0, [r1], #4` means performing a **post-indexing** operation. This means increasing the value of the base register by offset, after performing the load. We can thing of this as being analogous to the `++` syntax available in C. This also saves an instruction compared to RISC-V.

As we can see, the main point of this complication is to save a few instructions when accessing arrays, or generally where striding memory access patterns are needed. Since a word is 4 bytes on this machine, this is useful for walking through an array or copying a block of memory.

For example:

```
copy:
    ldr r3, [r0], #4
    str r3, [r1], #4
    b   copy
```

copies one word at a time while automatically advancing both pointers.

2.5 Loading constants and addresses

Assembly code also needs mechanisms for loading constants, referring to addresses, defining labels, and changing the control flow.

Let's have a table of example (pseudo and normal) instructions for constant loading.

Assembly	C equivalent
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;`
`mov sp, r0`	`sp = r0;`
`movs r3, #0`	`r3 = 0;`
`copy_data:`	label: a name for this address
`b copy_data`	`goto copy_data;`
`bl main`	`main();`

The pseudo-instruction

```
ldr r0, =_estack
```

loads the value represented by `_estack` into `r0`. This is often an address, but the syntax can be used for arbitrary 32-bit constants.

A large constant generally cannot be encoded directly in a single Thumb instruction. The assembler can therefore place the constant in a nearby **literal pool** and generate a PC-relative load.

For example, GDB may show:

```
ldr r0, [pc, #92]
```

The directive

```
.ltorg
```

asks the assembler to emit the current literal pool at that point.

Note that a label names an address in the program:

```
copy_data:
    ...
```

A branch transfers execution to that address:

```
b copy_data
```

This is analogous to a C `goto`, although assembly labels are addresses in the instruction stream rather than variables.

The instruction `bl` means **branch with link**. It branches to the target and simultaneously saves the return address in the **link register (LR)**. This is the normal mechanism used for a subroutine call:

```
bl main
```

A corresponding return normally uses the address stored in `lr`.

2.5.1 Branches

Assembly	C equivalent
`cmp r1, r2`	compute `r1 - r2` and update the flags
`bhs done`	`if (r1 >= r2) goto done;` `// unsigned comparison`
`orr r1, r1, #0x10`	`r1 |= 0x10;`
`cpsid i`	disable interrupt acceptance
`cpsie i`	enable interrupt acceptance

Tabella 3: Conditional branches, bit operations and interrupt control.

`cmp` performs a subtraction internally (`r1 - r2`) but does not store the result. Instead, it updates the condition flags (`N`, `Z`, `C`, and `V`) in the xPSR.

A subsequent conditional branch examines these flags.

For example, `bhs` means "higher or same" for an **unsigned** comparison:

```
cmp r1, r2
bhs done
```

This corresponds to:

```
if (r1 >= r2)
    goto done;
```

A C `while` loop therefore naturally becomes a test at the top, followed by a backwards branch at the bottom:

```
loop:
    cmp r1, r2          @ while (r1 < r2) {
    bhs done
    ...                 @     body
    b   loop            @ }
done:
```

2.5.2 Interrupt masking

On Cortex-M, `cpsid i` disables the acceptance of maskable interrupts, while `cpsie i` enables them again:

```
cpsid i
...
cpsie i
```

These correspond conceptually to the CMSIS functions `__disable_irq()` and `__enable_irq()`.

2.5.3 Assembler directives

Assembly source contains two fundamentally different kinds of lines:

1. **Instructions**, which become CPU instructions.
2. **Assembler directives**, which control how the assembler constructs the object file.

Directives generally begin with a period (`.`) and do not themselves execute on the CPU.

For example:

```
.section .isr_vector,"a",%progbits
.global isr_vector

isr_vector:
    .word _estack
    .word Reset_Handler

.section .text,"ax",%progbits
.global Reset_Handler
.thumb_func

Reset_Handler:
```

Here:

- `.section` selects or creates an ELF section.
- `.global` makes a symbol visible to the linker and other object files.
- `.word` emits a 32-bit value into the current section.
- `.thumb_func` tells the assembler that the following function is Thumb code.

Thus, `isr_vector` is not a function being executed by the CPU: the assembler is simply placing two 32-bit values into the `.isr_vector` section.

The corresponding idea in C is:

```
__attribute__((section(".isr_vector")))
const uint32_t isr_vector[] = {
    (uint32_t)&_estack,
    (uint32_t)Reset_Handler
};
```

Without `.global`, an assembly symbol is local to the object file, similar in effect to a `static` C symbol.

2.5.4 Thumb function addresses

Cortex-M executes Thumb instructions exclusively. Function pointers and exception-vector entries therefore have bit 0 set to indicate Thumb state.

For a symbol such as `Reset_Handler`, the assembler/linker can produce an address with bit 0 set when it is declared as a Thumb function:

```
.thumb_func
Reset_Handler:
```

For example, if the actual instruction address is `0x00000040`, the Thumb function pointer is represented as `0x00000041`.

The CPU uses this bit to identify the target as Thumb code; the actual instruction address is still aligned to an even address.

2.6 Faults

Let's look in more detail at **faults**, so that we can understand the whole of the *handlers* defined in the ISR vector table.

An **exception** is any event that interrupts the current code and causes the CPU to execute a handler whose address is stored in the vector table, switching to Handler mode.

#	Exception	Raised by
0	*None*	Slot holds the initial SP
1	Reset	Power-on, reset button
2	NMI	Non-maskable interrupt
3	**HardFault**	Faults that cannot be handled elsewhere
4	**MemManage**	Memory protection violations
5	**BusFault**	Bus access errors
6	**UsageFault**	Invalid instruction use, division by zero, ...

#	Exception	Raised by
7–10	*Reserved*	Slot must hold `0`
11	SVCall	The `svc` instruction
12	DebugMonitor	Debug events (unused here)
13	*Reserved*	Slot must hold `0`
14	PendSV	A software request
15	SysTick	Built-in timer
16+	IRQs	Device interrupts

At reset, only HardFault is active among the configurable fault handlers: faults that cannot be handled by their corresponding handlers escalate to HardFault. This corresponds to "escalate 3" behavior observed during debugging.

2.6.1 Handlers

The initial vector table contains only two words, but the Cortex-M processor expects one word per exception vector, for a total of 16 entries in the initial table. Slots 7–10 and 13 are reserved and must contain zero. Every other slot must contain a valid handler address, even if no specific handler has been implemented yet.

A default handler can be defined as follows:

```
    .word HardFault_Handler         @ slot 3, and so on up to 15

    .weak      HardFault_Handler
```

```
    .thumb_set HardFault_Handler, Default_Handler

Default_Handler:
    b Default_Handler                @ for (;;) {}
```

The equivalent declaration in C is:

```
void HardFault_Handler(void)
    __attribute__((weak, alias("Default_Handler")));
```

A **weak symbol** provides a default implementation that can be overridden by a normal definition in another object file. If `main.c` defines its own `HardFault_Handler`, the linker uses that implementation instead of the default.

The completed vector table occupies 16 words:

$$16 \times 4 = 64 \text{ bytes} = \texttt{0x40}.$$

Consequently, if the table starts at address `0x0`, the `Reset_Handler` entry point moves from `0x8` to `0x40`. Since Cortex-M executes Thumb instructions, its vector entry contains `0x41`, with bit 0 set to indicate Thumb state.

2.7 CPU registers

One interesting thing about ARM is that some CPU settings live in *registers* which are stored at certain **memory addresses**.

For instance, the **SCB** (*System Control Block*) is a set of 32-bit registers inside the CPU, reached like memory in the system area at `0xE000E000` – `0xE000EFFF`.

Let's recall the default processor registers:

Register	Purpose
`r0–r12`	General-purpose registers
`r13 (SP)`	Stack pointer
`r14 (LR)`	Link register; return address
`r15 (PC)`	Program counter
`xPSR`	Flags, execution state and exception number
`MSP`	Main stack pointer
`PSP`	Process stack pointer
`PRIMASK`	Masks configurable interrupts
`BASEPRI`	Masks interrupts below a priority threshold
`FAULTMASK`	Masks exceptions except NMI
`CONTROL`	Selects stack and privilege state in Thread mode

The exact register set depends on the Cortex-M implementation. In particular, `BASEPRI` is not implemented on every Cortex-M core.

The `SP` register refers to the currently selected stack pointer, either `MSP` or `PSP`. The processor uses the main stack pointer in Handler mode.

These peripherals expose registers through fixed memory addresses. They are accessed using ordinary load and store instructions.

Let's look at the main peripherals we have:

Block	Base address	Purpose
SCnSCB	`0xE000E000`	System control and feature registers
SysTick	`0xE000E010`	24-bit system timer
NVIC	`0xE000E100`	Interrupt enable, pending state and priorities
SCB	`0xE000ED00`	Exceptions, faults and processor configuration
MPU	`0xE000ED90`	Memory protection (optional)
DWT	`0xE0001000`	Watchpoints, cycle counter and profiling
FPB	`0xE0002000`	Flash patch and breakpoint unit
ITM	`0xE0000000`	Instrumentation and debug output
CoreDebug	`0xE000EDF0`	Debug control and status
TPIU	`0xE0040000`	Trace output interface

These addresses correspond to the Cortex-M3 memory map. Optional blocks may be absent or incompletely implemented in a particular processor or emulator.

The *System Control Block* (**SCB**) provides registers for processor configuration, exception handling and fault diagnosis.

Let's look at some important elements of the SCB:

Register	Offset	Purpose
`CPUID`	`0x00`	Processor identification
`ICSR`	`0x04`	Exception and interrupt state
`VTOR`	`0x08`	Vector table base address
`AIRCR`	`0x0C`	System reset and priority configuration
`SCR`	`0x10`	Sleep behavior
`CCR`	`0x14`	Processor configuration
`SHPR1-3`	`0x18-0x23`	System exception priorities
`SHCSR`	`0x24`	System handler enable and status
`CFSR`	`0x28`	Configurable fault status
`HFSR`	`0x2C`	HardFault status
`DFSR`	`0x30`	Debug fault status
`MMFAR`	`0x34`	Memory-management fault address
`BFAR`	`0x38`	Bus fault address

2.8 Real chips

QEMU's CPU is ready as soon as it leaves reset. A real MCU's startup code usually does more, often in a vendor function `SystemInit()`:

- **Clock / PLL**: the chip starts on a slow internal oscillator; the PLL multiplies the clock frequency up to the desired operating frequency.
- **Flash wait states**: Flash memory is slower than a fast CPU. The Flash controller must be configured to insert additional wait cycles *before* increasing the CPU clock frequency.
- **Watchdog**: a timer that resets the chip unless software periodically "feeds" it. On some chips the watchdog starts running immediately after reset, so the startup code must feed or disable it.
- **FPU (Cortex-M4F)**: the floating-point unit is disabled at reset. It can be enabled through the `CPACR` register at `0xE000ED88`; otherwise executing an FPU instruction can cause a UsageFault.

- **C++**: before entering `main()`, the startup code must call the static constructors stored in `.init_array`.

3 Lesson 01-10-26

3.1 Freestanding C

The C standard defines two kinds of environment:

- **Hosted**: C library (`libc`) and an OS sit under the user code, library functions such as `prinf()`, `malloc()`, `fopen()` are available. `main()` is called by the `libc` startup code, called by the OS. Of course, each OS implementation implements its own `libc` (GNU/Linux, Windows, ...). There are even versions of `libc` which run on bare metal (`newlib`);
- **Freestanding**: nothing stis under the user code, no library functions are available and nobody calls `main()` (the default entry point is the reset handler).

We will be using a *freestanding* environment from now on.

System calls

In fact, `libc` in Linux is only an interface between user code and the OS: a call to a `libc` function in Linux means:

- Performing a little setup work in userland, via the `libc` implementations;
- Performing some kind of *system call*, which leaves the CPU to the kernel;
- Waiting for the kernel to carry out our system call;
- Getting the result back from `libc`.

We decide to opt out of libraries like `libc` because they still need architecture-specific hooks (`write`, `crt0`, ...) hooks to be implemented: we're better off doing everything ourselves.

Compiler helpers

One piece of help we will use is the *compiler helpers* provided by the **GCC** (*Gnu Compiler Collection*) toolchain we use. These implement some basic C operators (such as divide and modulo), which may not be implemented in hardware. Was someone to be curious, some example helpers can be found at `https://github.com/seggiani-luca/micro-sim/blob/main/eprom/src/lib/helpers.cpp` (there are likely cleaner and better implemented one somewhere in the GCC source).

3.1.1 Semihosting

We still need to be able to print stuff to console. The solution we will use is **semihosting**: the target asks the *host* (our machine) to perform the I/O.

1. The program puts an operation code in `r0` (`0x04` = `SYS_WRITE0`) and the string address in `r1`. `SYS_WRITE0` is the opcode for "print this NULL-terminated string";
2. It executes `bkpt 0xab`: a breakpoint instruction with a special number;
3. QEMU (or a debugger) sees the breakpoint, reads `r0` and `r1`, reads the string from target memory, and prints it **on your laptop**;
4. The program resumes after the `bkpt`.

This only works with a debugger or emulator attached (e.g., `-semihosting` in `make run`). On a real board without a debugger, `bkpt` would simply stop the CPU.

In code

In code, this translates to:

```
static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address
    */
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}
```

Let's look at what `bkpt 0xab` really does:

- The program **stops** doing its own work;
- Control passes to **someone more powerful** (QEMU, outside the CPU);
- That someone performs the job and **hands control back**.

This is the same shape as a system call on Linux: your program cannot drive the terminal itself, so it **asks the kernel** to do it.

Next question: can the Cortex-M3 itself have a 'powerful" part and a 'restricted" part, like kernel mode and user mode? **Yes**: privileged and unprivileged execution, next slide.

3.2 Privilege level

Even if our implementation of an OS will be *flat*, let's go over **privilege** levels in our ARM Cortex-M3 CPU.

W know that an *OS kernel* runs in kernel mode, where as an applications runs in *user mode*. Cortex-M3 has the same idea in hardware:

- **Handler mode**: used when running an exception handler, always privileged;
- **Thread mode**: normal mode used for user code, can be privileged or unprivileged.

After reset we find the CPU in thread mode, privileged. We have to manually set the CPU to unprivileged thread mode if we want protection on user code.

A special CPU register (not in memory) selects Thread-mode privilege:

Bit	Name	Meaning
0	**nPRIV**	0 = privileged, 1 = unprivileged (Thread mode only)
1	SPSEL	selects which stack pointer Thread mode uses (not used here)

Special registers are reached with two dedicated instructions:

Instruction	C equivalent	
`mrs r0, control`	`r0 = CONTROL;`	(read special register)
`msr control, r0`	`CONTROL = r0;`	(write special register)

3.2.1 Dropping privilege

So, for example, to drop our privilege lever to unprivileged:

```
mrs r0, control     @ r0 = CONTROL
orr r0, r0, #1      @ r0 = r0 | 1       (set nPRIV)
msr control, r0     @ CONTROL = r0      -> now unprivileged
isb                 @ discard instructions already fetched under old rules
```

Writing CONTROL is itself a privileged operation: an unprivileged `msr control, ...` is ignored. Code cannot promote itself back. The only ways back to privileged: reset, or an exception (its handler runs in Handler mode, always privileged).

Attempt	Result
Read/write the **System Control Space** (`0xE000E000 ...`): SysTick, NVIC, SCB (VTOR, SHCSR, CCR, priorities)	**BusFault**
`msr control, r0` (raise privilege back)	ignored
`cpsid i` / `cpsie i` (mask interrupts)	ignored
Semihosting `bkpt 0xab`	QEMU refuses it by default → fault

Ordinary RAM, Flash, and user code still work normally. The restricted code simply cannot **reconfigure** the machine or silence it.

3.2.2 Raising privilege

Unprivileged code cannot touch the hardware, but it can still *call* privileged mode handlers: `svc #n` (*SuperVisor Call*) is allowed in unprivileged code.

1. It raises the **SVCall exception** (vector-table entry 11, `SVC_Handler`). The CPU enters **Handler mode**, and the corresponding handler runs privileged;
2. When the handler returns, the CPU goes back to the instruction after `svc`, still **unprivileged**.

`#n` is a small number encoded in the instruction. Real kernels use a request number and arguments to tell which service is wanted; here we only use `svc #0`.

This allows us to implement what effectively are **system calls**.

4 Lesson 07-10-26

4.1 ARM hardware interface

The ARM hardware interface is split in 2 components:

- The **CMDSDK** (*Cortex-M System Design Kit*): it is an hardware specification of ready-made peripheral designs that chip designers place next to the CPU;
- The **CMSIS** (*Cortex-M Software Interface Standard*): it is a standard for the C headers and functions that describe a chip.

Our board's peripherals are CMSDK blocks: Arm documents their registers once, the same on every chip that uses them. More specifically, it is an *MPS2*: an Arm development board built around an FPGA (a chip whose logic is loaded at power-on). *AN385* (Application Note 385) is one FPGA image for it: a Cortex-M3 plus CMSDK peripherals.

Address	Device
`0x4000_0000 / 0x4000_1000 / 0x4000_1000`	Timer 0, Timer 1, dual timer
`0x4000_4000 / 0x4000_5000 / 0x4000_6000`	UART0, UART1, UART2
`0x4000_8000 / 0x4001_0000`	Watchdog, GPIO pins
`0x4002_8000`	FPGA I/O (MPS2 block)

We use no vendor CMSIS files (no libraries at all): our register header is hand-written in CMSIS style, so its names look like those in vendor code.

Basically, we have a CMDSDK peripheral set, for which we program a CMSIS interface by hand.

4.1.1 Buses

A **bus** is the set of wires (address, data, control) that carries every load and store from the CPU to a memory or a device. Arm defines two kinds:

- **AHB** (*Advanced High-performance Bus*): the main bus; pipelined, one transfer per cycle. Memories and devices that need speed sit here;
- **APB** (*Advanced Peripheral Bus*): simple, slow, low power. For registers that are touched rarely: configure a timer, send one byte

The bridge turns an AHB access into an APB access: a few extra cycles are needed for conversion. For our code, nothing changes: a register is a load or store at an address on either bus; only the timing differs.

4.1.2 External interrupts

A device can raise an interrupt request (**IRQ**) line to tell the CPU "something happened". Device line N is served by vector-table slot $16 + N$, right after the 16 core slots we saw were related to internal interrupts (exceptions, such as the Reset Handler).

IRQ	Interrupt source	IRQ	Interrupt source
0	UART 0 receive	16	GPIO 2 combined
1	UART 0 transmit	17	GPIO 3 combined
2	UART 1 receive	18	UART 3 receive
3	UART 1 transmit	19	UART 3 transmit
4	UART 2 receive	20	UART 4 receive
5	UART 2 transmit	21	UART 4 transmit
6	GPIO 0 combined	22	SPI 2
7	GPIO 1 combined	23	SPI 3, SPI 4
8	Timer 0	24	GPIO 0, pin 0
9	Timer 1	25	GPIO 0, pin 1
10	Dual Timer	26	GPIO 0, pin 2
11	SPI 0, SPI 1	27	GPIO 0, pin 3
12	UART overflow (UART 0, 1, 2)	28	GPIO 0, pin 4
13	Ethernet	29	GPIO 0, pin 5
14	Audio I2S	30	GPIO 0, pin 6
15	Touch Screen Controller	31	GPIO 0, pin 7

4.2 ARM ABI

Let's look at the ARM **ABI** (*Application Binary Interface*), as it will be useful to know how to implement interrupt handlers (we have to know how arguments are passed around).

C and assembly can call each other only if both follow the same rules: the **AAPCS** (*Procedure Call Standard for the Arm Architecture*).

Register	Role in a call	Kept across the call?
r0–r3	The first four arguments; r0 also carries the result	No: **caller-saved** (scratch)
r4–r11	The function's own long-lived values	Yes: **callee-saved**
r12	Scratch	No: **caller-saved**
lr	Return address, written by `bl`	No: a function that calls another saves it first
sp	Must be a multiple of 8 at every call	Yes

- **Caller-saved**: if the caller still needs r0–r3 or r12 after the call, it saves them itself;
- **Callee-saved**: a function that wants to modify r4 must restore it before returning.

5 Lesson 14-10-26

Last week we built a small kernel: unprivileged code calls the kernel through system calls (`svc`), and only the kernel touches the hardware. This week's starting code runs two tasks, A and then B, both unprivileged, each with its own stack. But every byte of RAM is still shared: the kernel's variables, task A's and task B's sit side by side, and any of them can write anywhere.

5.1 Memory

In this lesson we start covering memory management. Memory is one shared resource, and several programs want it at once. The OS has four jobs:

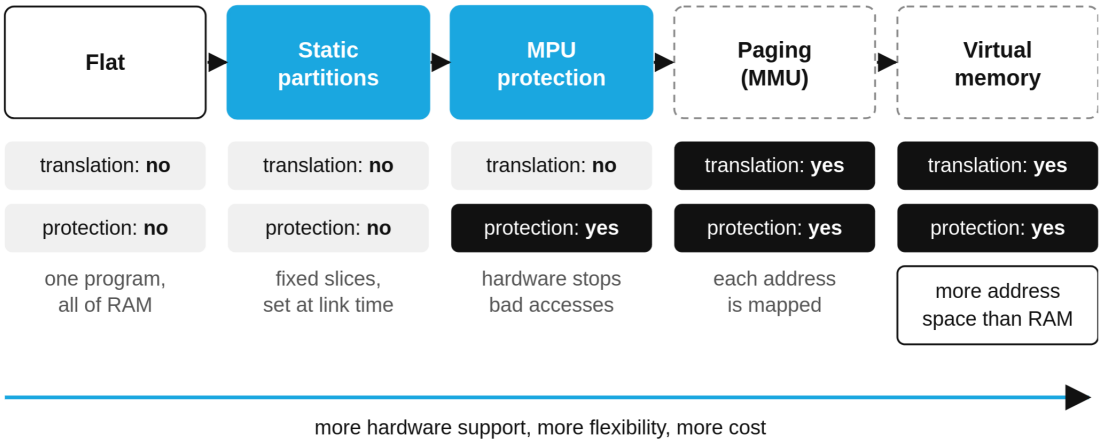
- **Placement**: decide where each program's code, data and stack go, so they do not overlap;
- **Sharing**: let programs use the same memory when they should (one copy of a library, a buffer passed to a device);
- **Protection**: stop a program from reading or writing memory that is not its own, by mistake or on purpose;
- **More programs than memory**: when the programs together need more memory than there is, keep only the parts in use in RAM.

There are a number of mechanisms used to perform memory management and protection.

5.1.1 Translation and protection

First, we should distinguish between **translation** and **protection**.

- **Translation** means the address the program uses is changed into another address before it reaches memory;
- **Protection** means the hardware stops accesses that are not allowed.



We now present an evolution timeline of memory management techniques.

- **Flat**: one program, or a kernel and its tasks together, share all of RAM. No translation, no protection;
- **Static partitions**: each task gets a fixed slice of RAM, set at link time. Still no protection: nothing stops a task from writing outside its slice;
- **MPU protection**: a hardware unit checks every access against a list of allowed regions and stops bad ones. Protection, but no translation: an address means the same thing for everyone;
- **Paging (MMU)**: hardware maps each address the program uses to a different physical place. Translation and protection;
- **Virtual memory**: paging plus the disk, so programs can use more memory than the RAM has.

Note that each step needs more hardware and buys more flexibility.

6 Lesson 21-10-26

6.1 Context switching

This is likely better explained in code.

6.2 Real-time scheduling

Let's now focus on how real-time systems define **schedulers**.

We first focus on what *real-time* means. Most programs only have to be correct: given the input, compute the right output, whenever it is ready. A real-time system must be correct and *on time*: it reacts to an event from the outside world, and the result must be ready within a fixed time, the deadline.

We saw there were categories of real-time systems:

- **Hard**: if the task deadline is missed, catastrophic consequences on the system or its environment;
- **Firm**: the result is useless and is thrown away, but no serious damage;
- **Soft**: the result is still useful, only worse; meeting the deadline is desirable.

A fast system minimises the average time a set of tasks takes: (average turnaround, average waiting). A real-time system guarantees the timing of each task: every job must finish by its own deadline, even in the worst case. A low average says nothing about one task: one late job can hide behind many early ones. A faster CPU helps, but it is not a guarantee: what matters is which task runs when, that is the scheduling. So a real-time system values **predictability**: we must be able to compute, in advance, when each task finishes.

6.2.1 Types of tasks

Real-time system discussion benefits from a quick overview on **task types**. There are three:

- **Periodic**: a new job is released at a constant rate, every period T. Also called time-driven. Easy to reason about (and do maths on), also quite similar to how most embedded systems are structured: every T seconds the system should for instance check input, execute an algorithm, update a value, repeat ...
- **Aperiodic**: jobs are released by events, at irregular times, with no known minimum gap: in theory two can arrive at the same instant. This is harder to handle than the one above;
- **Sporadic**: aperiodic, but two consecutive jobs are always at least a minimum inter-arrival time apart. In the worst case, a sporadic task behaves like a periodic task with T. These are easier to reason about if we assume them to not be that frequent and lift the hard real-time assumption slightly.

We will use a simplified model for tasks.

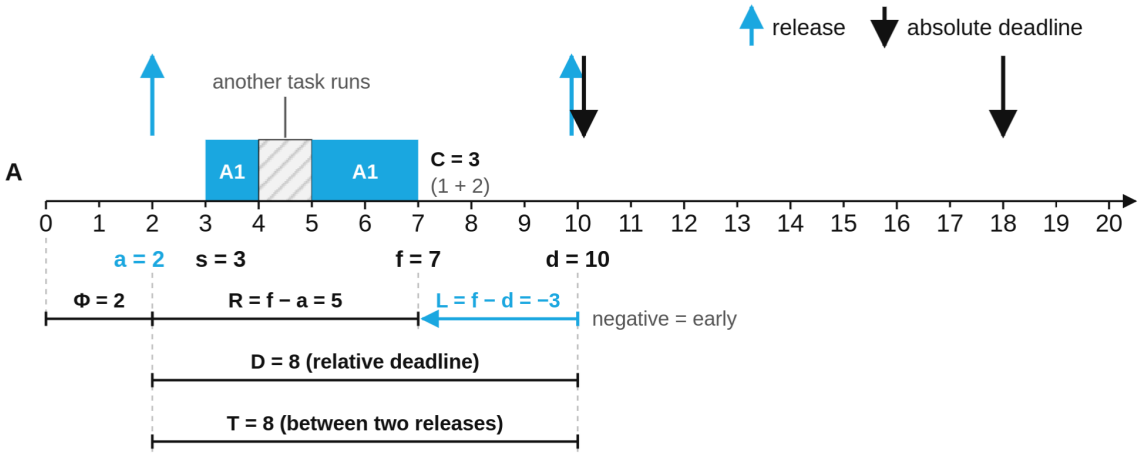
- We have a *fixed number* of tasks, all known in advance;
- Every task is periodic, with a known period T;
- The tasks are independent: they share no resources and do not wait for each other (no precedence);
- Each task has a hard deadline equal to its period: a job must finish before the next job of the same task is released ($D = T$);
- Each task has a known, fixed worst-case execution time C. This can be obtained in practice via either static measuring (before uploading programs to the OS) or dynamic measuring (by the OS itself).
- There is one CPU, and all the kernel's overheads, for example the context switch, are negligible.

6.2.2 Scheduling metrics

Let's introduce some **metrics**.

- **Release** (arrival, activation) time a: the job becomes READY. Job k is released at $a = \phi + (k-1) \times T$;
- **Phase** ϕ : the release time of the first job. $\phi = 0$ if not given;
- **Computation** time C: the CPU time the job needs; we use its worst case, the **WCET** (*Worst-Case Execution Time*);
- **Period** T: the time between two releases.
- **Relative deadline** D the time the job has after its release; $D = T$ here;

- **Absolute deadline** $d = a + D$: the instant the job must be finished by. With $D = T$, it is the next release;
- **Start time** s, **finish time** f. In general $f \neq s + C$, because the job can be preempted;
- **Response** time $R = f - a$: from release to the end. A job meets its deadline when $R \leq D$;
- **Lateness** $L = f - d$: negative means early, 0 means just in time, positive means deadline missed.



The response time $R = f - a$ is sometimes called *turnaround*, and $s - a$ the response time. In real-time systems, we stick to the above convention.

It is also extremely useful to define the **utilization** of the CPU as:

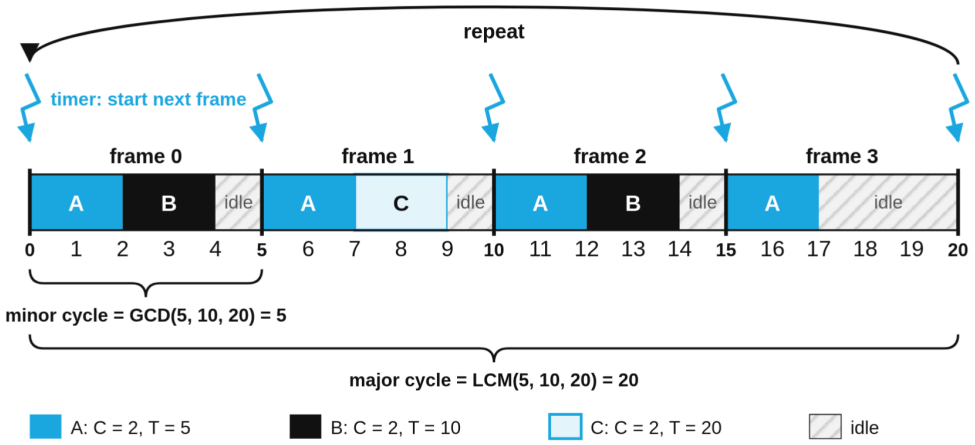
$$U = \sum_{i}^{N} \frac{C_i}{T_i}$$

That is the sum of all execution time over period ratios for all tasks.

Lastly, we define the hyperperiod H as the least common multiple of all the task periods. With all phases at 0, every task releases a job at 0 and again at H, so the pattern of releases repeats at every H.

6.2.3 Timer scheduling

Some embedded applications are simple: a few periodic tasks that do not interact, for example reading sensors through one shared ADC and updating outputs. They do not need a full operating system. **Timeline** scheduling, also called a cyclic executive: the whole schedule is computed offline, before the system runs, and stored as a fixed table. The time axis is divided into slots of equal length, the *minor cycles* (or frames), equal to the greatest common divisor of the periods. Each frame runs a fixed list of jobs, one after the other, without preemption, and a timer interrupt marks the start of every frame. The sequence of frames that then repeats identically is the *major cycle*, equal to the least common divisor. At run time there is no scheduler deciding anything: the program follows the table.



7 Lesson 28-10-26

We spent the last lessons implementing a **round-robin** scheduler for our OS. This is all better explained in code than in notes.

Today we will do a theory-only lesson about scheduling algorithm The task model is the same from the previous lesson.

7.1 Fixed priority

7.1.1 RM scheduling

RM (*Rate Monotonic*) is a *fixed priority* scheduling algorithm. Fixed in this case means that each task gets a priority once, before the system starts, and keeps it forever (or until it terminates, in case of a process).

In rate monotonic, the task priority is:

$$\text{priority} = \frac{1}{t}$$

where T is the task period, with appropiate tie rules. It is *preemptive*: a released job with a shorter period preempts the running one.

7.1.2 Response-time analysis

RM scheduling is *optimal* among fixed priority scheduling algorithms.

We know how to get a sufficient (but not necessary) condition for the feasibility of RM scheduling on a set of tasks, which is from *Liu-Layland* (`https://raw.githubusercontent.com/seggiani-luca/eosa-notes/2f281524d2b6b70b26ef55339e762293d97a5d43/notes/liu-layland/main.pdf` is a sketch of the proof from my notes). The bound formula we get from it is:

$$U < n(2^{\frac{1}{n}} - 1)$$

of the utilization U for n tasks.

A techique we want to give now is **RTA** (*Response-Time Analisys*). Let R_i be the time a tasks takes to do its processing, including the interference from other tasks. This is just our response time $R = f - a$ with f the termination time and a the period edge.

We can define R_i recursively for each task:

$$R_i = C_i + \sum_{j\,:\,\text{prio}(j)>\text{prio}(i)}^{n} \left\lceil \frac{R_i}{T_j} \right\rceil \cdot C_j$$

We know that $\frac{R_i}{T_j}$ is simply the time task j is dispatched in the execution window R_i of the current task.

Then this becomes simply the time the task takes to execute by itself, plus the time all higher priority tasks "steal away" from it, every time they are scheduled in its execution window.

The way we evaluate this is by starting with $R_i(0) = C_i$ for each task. Then we update:

$$R(k+1) = C_i + \sum_{j\,:\,\text{prio}(j)>\text{prio}(i)}^{n} \left\lceil \frac{R_i(k)}{T_j} \right\rceil \cdot C_j$$

Note that the highest priority task task R_n has no interference:

$$R_n = C_n$$

so the response time is the same after the upate.

For all other tasks the update changes values, and we stop the iteration when we reach a fixed point $R_i(k+1) = R_i(k)$ for all tasks. The task set is thus schedulable when we can say:

$$R_i \leq D_i, \quad \forall i = 1, 2, ..., n$$

So each task respects its deadline.

7.2 Dynamic priority

7.2.1 EDF scheduling

Let's move on to **EDF** (*Earliest Deadline First*) scheduling. This is a *dynamic priority* scheduling algorithm, which means the priority belongs to the job, not to the task: two jobs of the same task have different deadlines. IF priorities are dynamic the ranking between two tasks can change over time.

The priority in earliest deadline first is that at every instant, we run the ready job with the earliest absolute deadline:

$$d = a + D$$

where a is the last dispatch instant.

This algorithm is preemptive too: a released job with a strictly earlier deadline preempts the running one.

The kernel overhead is that at each release the kernel must compute d and keep the ready jobs ordered by d (higher than RM).

EDG scheduling is *optimal* among dynamic priority scheduling algorithms. We have a simple schedulability bound (also from Liu-Layland):

$$U < 1$$

7.3 Aperiodic jobs

Let's now focus on **aperiodic** jobs. An aperiodic job arrives once, at a time that is not regular: a key press, a message from the network, an alarm. It has an execution time C and, often, a deadline D.

We don't know a T period, as we did before: we only know the arrival time a.

The algorithms will be:

- For **only one-shot sets**: the goal is to meet the deadlines, or be as little late as possible. Algorithms: Jackson and Horn;
- For **mixed sets**: hard periodic tasks plus aperiodic jobs whose arrival is not known in advance. The goal is to keep every periodic deadline and serve the aperiodic jobs fast. Algorithms: background, polling server, deferrable server.

The *lateness* of a task is simply $L = f - d$, that is the time past the deadline the task took to complete. Ideally we want this negative.

7.3.1 EDD scheduling

This is *Jackson*'s algoritm, known as **EDD** (*Earliest Due Deadline*).

The problem is given by the triple:

$$(1, \text{sync}, L_{\max})$$

which stands for one processor, n independent jobs that all arrive at 0, each with its own Cc and D. The goal is to minimise $L_{\max}$.

Earliest Due Date: run the jobs one after the other in order of non-decreasing deadline. All jobs are there at 0: preemption cannot help, so EDD runs each job to the end. The order is fixed before the start: sort the jobs by d.

Jackson's theorem states that EDD is optimal for Lmax: no other order gives a smaller maximum lateness . So if EDD misses a deadline ($L_{\max} > 0$), every order misses one: the set is infeasible.

7.3.2 EDF over jobs scheduling

This is *Horn*'s algoritm, known as **EDF** (*Earliest Deadline First*) over jobs.

The problem is given by the triple:

$$(1, \text{preempt}, L_{\max})$$

which stands for one processor, n independent jobs that arrive at any time, each with its own Cc and D. The goal is to minimise $L_{\max}$.

This is Horn's rule: at every instant, run the ready job with the earliest absolute deadline $d = a + D$. A new job with a strictly earlier deadline preempts the running one.

This is EDF, applied to one-shot jobs instead of periodic tasks. Horn's theorem states that EDF is optimal for $L_{\max}$: the same swap argument as Jackson, applied to the slots of the schedule. Preemption is needed: a job with a near deadline may arrive while a long job runs. Without preemption, the problem with arbitrary arrivals is much harder. With all arrivals at 0 there is no preemption, and Horn gives the same schedule as Jackson.

8 Lesson 29-10-26

We hit the ground running from the previous lesson, continuing to focus on real time scheduling algorithms for real time systems.

8.1 Mixed job sets

We understood that real systems mix both kinds of work:

- **Periodic tasks**: time-driven, critical control loops with hard deadlines;
- **Aperiodic jobs**: event-driven (a button, a message, a fault); hard, soft or no deadline; arrival time not known in advance.

For mixed sets, let's forget the fact that aperiodic jobs have deadlines and assume that aperiodic jobs are served first come, first served (**FCFS**) among themselves (same arrival: lowest identifier). For periodic tasks let's stick to **RM** (7.1.1).

Three ways to serve them: in **background**, with a **polling server**, and with a **deferrable server**.

8.1.1 Background scheduling

In **background** scheduling the periodic tasks run with *RM*, an aperiodic job runs only when no periodic job is ready. This is the simplest to implement (requires basically no implementation) and focuses on periodic jobs as first-class.

We keep two ready queues: periodic (RM priority) and aperiodic (FCFS). The aperiodic queue is served only when the periodic one is empty. A periodic release preempts the aperiodic job at once (focus on periodic jobs). The periodic schedule is exactly the RM schedule without the aperiodic jobs: we only use the free slots left by RM if possible. This means the aperiodic response is the time that is left: it depends on the periodic load.

The problem of background scheduling is this last point: we know that in fact aperiodic tasks have deadlines, and with this approach we ignore them. This means that if the periodic task set utilizes the CPU fully (or almost fully) we might get serious aging problems of the aperiodic tasks, if not not executing them at all.

8.1.2 Polling server

A server is a periodic task created to run aperiodic jobs. It has a period (like any periodic task) T_s and a budget (capacity) C_s: the time it may use in each period. Its utilisation is $U_s = \frac{C_s}{T_s}$.

The **polling server** is scheduled with **RM** like any other periodic task: its priority comes from T_s. If T_s equals the period of a task, both have the same RM priority: the task runs first.

At each period start the budget si set to C_s. It never accumulates. The server runs only when it has budget and an aperiodic job is pending; each slot it runs costs 1 unit. Now:

- If no job is pending at its activation (the period start): the budget is lost (0) until the next period;
- If the queue becomes empty while it has budget left: the rest of the budget is lost;
- A job that arrives during a period is served in that period only if the budget is still there (a job was pending at the activation and the queue has not emptied yet); otherwise it waits for the next period (because the budget is lost).

There's a feasibility bound for the polling server:

$$P = \prod_{i}^{n}(U_i + 1) \leq \frac{2}{U_s + 1}$$

Where U_s refers to the server utilization given maximum capacity C_s:

$$U_s = \frac{C_s}{T_s}, \quad U_i = \frac{C_i}{T_i}$$

This bound is only *sufficient*, necessary (see 7.1.2 or the Liu-Layland notes). In fact, this is basically Liu-Layland including the server task at maximum capacity:

$$\prod_i^n (U_i + 1) \cdot (U_s + 1) \leq 2$$

which is also valid.

Proof
We can prove that the Liu–Layland utilization bound given in 7.12 or in the notes implies the product condition. The two conditions are not equivalent in general; the utilization bound is a sufficient condition for the product inequality:

$$\sum_i^n U_i \leq n\left(2^{\frac{1}{n}} - 1\right) \implies \prod_i^n (U_i + 1) \leq 2$$

This proof seems to come easy using the AM-GM inequality (or *Jensen*'s inequality if you fuck around long enough). Since we seemed to have fucked around long enough, in this proof we try to cover both bases.

First we start from the fact that:

$$\frac{1}{n}\sum_i^n \log\left(U_i + 1\right) \leq \log\left(\frac{1}{n}\sum_i \left(U_i + 1\right)\right)$$

This comes from observing that $\log(x)$ is a concave function. Because of this concavity, the the average of the function's values is no greater than the function evaluated at the average. Really, this statement is just Jensen's inequality (which generalizes to any function f that is concave).

What we can do is take exponents on both sides:

$$\begin{aligned}
e^{\frac{1}{n}\sum_i^n \log(U_i+1)} &\leq \frac{1}{n}\sum_i^n \left(U_i + 1\right) \\
\left(e^{\sum_i^n \log(U_i+1)}\right)^{\frac{1}{n}} &\leq \frac{1}{n}\sum_i^n \left(U_i + 1\right) \\
\left(\prod_i^n \left(U_i + 1\right)\right)^{\frac{1}{n}} &\leq \frac{1}{n}\sum_i^n \left(U_i + 1\right)
\end{aligned}$$

The last step is simply noticing at the LHS that:

$$\begin{aligned}
e^{\sum_{i=1}^n \log(U_i+1)} &= e^{\log(U_1+1)+\log(U_2+1)+\cdots+\log(U_n+1)} \\
&= e^{\log(U_1+1)} e^{\log(U_2+1)} \cdots e^{\log(U_n+1)} \\
&= (U_1 + 1)(U_2 + 1)\cdots(U_n + 1) \\
&= \prod_{i=1}^n (U_i + 1)
\end{aligned}$$

At this point, what we proved is basically the AM-GM inequality.

We can now take the LHS of the original equivalence:

$$\sum_i^n U_i \leq n\left(2^{\frac{1}{n}} - 1\right)$$
$$\frac{1}{n}\sum_i^n U_i \leq 2^{\frac{1}{n}} - 1$$
$$\sum_i^n \frac{U_i}{n} + 1 \leq 2^{\frac{1}{n}}$$
$$\sum_i^n \frac{U_i}{n} + \sum_i^n \frac{1}{n} \leq 2^{\frac{1}{n}}$$
$$\frac{1}{n}\sum_i^n (U_i + 1) \leq 2^{\frac{1}{n}}$$

The LHS of this is the RHS of the inequality we proved earlier. If this is true, then it's also true that:

$$\left(\prod_i^n (U_i + 1)\right)^{\frac{1}{n}} \leq 2^{\frac{1}{n}}$$
$$\prod_i^n (U_i + 1) \leq 2$$

The last step was the same as applying the AM-GM inequality, thus the whole ordeal.

Sometimes this multiplicatory bound is called the *hyperbolic bound* (e.g. Wikipedia), which is a tighter sufficient condition for schedulability than the one originally presented by Liu and Layland.

"*Tighter*" here means *less conservative*: the Liu-Layland proof (again, read the notes) assumes the worst case scenario (called the *critical instant*) where all tasks are scheduled at the same instant. The multiplicative bound doesn't consider just this particular condition, so where the Liu–Layland bound depends only on the total utilization, it retains information about the individual task utilizations. Consequently, the hyperbolic test can distinguish between task sets with the same total utilization but different distributions of utilization across tasks.

The implication means this: all sets schedulable according to the Liu-Layland bound are schedulable according to the hyperbolic bound. Some other sets schedulable according to the hyperbolic bound might be missed by the Liu-Layland bound.

□

This bound is useful for determining the polling server *capacity*, that is **sizing** the polling server. We solve the test for U_s: the largest server utilisation is:

$$U_{s\text{max}} = (2 - P)/P$$

Where P is the product of other tasks. As a rule of thumb:

- T_s is set equal to the shortest period: the server gets the highest priority (equal to that task, which runs first);
- $C_s = U_s \cdot T_s$ with $U_s \leq U_{s\text{max}}$; at the exam, the largest integer C_s.

8.1.3 Deferrable server

The polling server wastes its budget when no job is pending at the right moment, so a job that arrives just after waits a whole period. The **deferrable server** keeps the budget.

We assume the same parameters C_s, T_s, same RM priority from T_s (it loses a tie with a task of the same period). The difference is that it keeps its budget until the end of its period, even when no job is pending. It serves an aperiodic job at any time while budget is left: an arriving job can preempt a lower-priority task at once. At each period start the budget is set back to C_s: not added to what is left.

This is way better at servicing aperiodic tasks. The price is **back-to-back** execution: the deferrable server can use its budget at the end of one period and again at the start of the next. Up to $2C_s$ can be used in a row, without servicing any other task.

Because of this last consideration, we know that the deferrable server is not a plain periodic task: back-to-back it can run $2C_s$ in a row, so it needs its own test (sufficient only):

$$P = \prod_{i}^{n}(U_i + 1) \leq \frac{U_s + 2}{2U_s + 1}$$

The proof of this test requires first understanding the original Liu-Layland proof (again, notes). It seems kind of clunky, so it won't be notated fully here.

What we can say is that from the original proof we had recurrent relations to express bounds on execution times C_i. Specifically, for C_n we had:

$$C_n = 2T_1 - T_n$$

at full utilization.

To account for the fact that our deferrable server task can run twice its C_s in a row, straddling a period edge, we change this to:

$$C_n \leq KT_1 - T_n, \quad K = \frac{U_s + 2}{2U_s + 1}$$

The rest of the proof follows from this.

This bound is also useful for sizing the polling server:

$$U_{s\max} = \frac{(2 - P)}{2P - 1}$$

Same as above, we have a rule of thumb:

- T_s is set equal to the shortest period: the server gets the highest priority (equal to that task, which runs first);
- $C_s = U_s \cdot T_s$ with $U_s \leq U_{s\max}$; at the exam, the largest integer C_s.

8.2 Sources of unpredictability

Every test of this week assumes that C is a true upper bound (the *Worst Case Execution Time*, **WCET**) and that nothing else takes the CPU. A real system must be predictable: its timing can be bounded in advance. Sources of unpredictability around the CPU: DMA steals bus cycles; cache hit or miss varies execution time; interrupts cause unbounded preemption; system calls run in non-preemptive kernel sections; memory management adds paging and allocation delays. All make WCET and response time hard to bound.

- **DMA**: the DMA controller and the CPU share the bus. In burst mode the CPU waits for a whole block; in cycle stealing it waits one word at a time. Fix: a time-sliced bus (fixed slots for CPU and DMA): slower, but the task time no longer depends on the transfer;
- **Cache**: a hit is fast, a miss is slow. The average improves, the worst case is hard to compute; each preemption destroys locality and adds misses (one measure on a PowerPC: WCET +33%). Fix: disable or lock the cache, or a larger, safe WCET;
- **Interrupts**: an ISR preempts any task, and the number of interrupts in a window is hard to bound. Fix: a very short ISR that only wakes a task that does the work, scheduled with its own priority; or treat the device as aperiodic jobs served by a server, as in this unit;
- **System calls**: a non-preemptible kernel section delays even the highest-priority task. Their time must be known and bounded, and the kernel made preemptible;
- **Memory management**: paging and dynamic allocation take a time that depends on the state of memory. Real-time code allocates at start-up, avoids recursion and unbounded loops.

9 Lesson 04-11-26

9.1 Synchronization

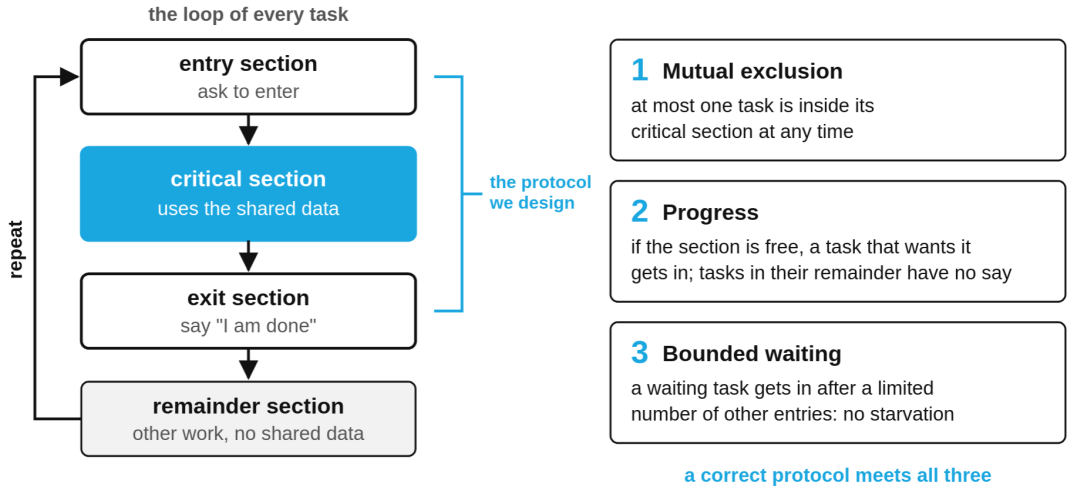
We no have to talk about **synchronization**.

An operation is **atomic** if it runs as one step that nobody can cut in two: no other task runs in the middle of it. Some of our operations have to be atomic, or at least be atomic in regard to some OS resource (say, a peripheral): have **mutual exclusion** on it.

Currently, we have no OS primitive to enforce this. Let's fix this.

- A **shared resource** is anything two or more tasks use: a variable, a buffer, a device such as the UART;
- A **race condition** is a bug where the result depends on the order in which the steps of the tasks happen to run;
- An **interleaving** is one possible order of all the steps of the tasks. With preemption, the scheduler can produce any interleaving;
- A **critical section** is a piece of code that uses a shared resource and must not run in the middle of another.

To remove the race, we do not need to forbid switches: we need to stop a second task from entering its critical section while the first one is inside, in regards to one common shared resource.



We see that whatever protocol we define must meet:

- **Mutual exclusion**: at most one task is inside its critical section at any time. This is the whole point of the protocol;
- **Progress**: if nobody is inside and some tasks want to enter, one of them gets in; a task busy in its remainder section cannot stop them. A protocol that keeps the door always closed is safe but useless;
- **Bounded waiting**: once a task asks to enter, the other tasks can enter only a limited number of times before it does. Otherwise it may starve.

We assume nothing about speed: the protocol must work for any interleaving, any quantum, any number of interrupts.

Let's see a few possible solutions.

9.1.1 Mask interrupts

On one CPU, a task loses the CPU only in two ways: an interrupt (the tick, then PendSV), or its own call (`yield()`, `sleep()`, ...). If the critical section masks interrupts and makes no such call, no other task can run inside it.

This, though, is quite limited and applies to small things we basically alerady saw, *inside* the kernel:

- **Privileged only**: `cpsid i` changes PRIMASK only in privileged mode. Our tasks are unprivileged, and in unprivileged mode the instruction is simply ignored: no fault, no effect. That is on purpose: a task that could mask interrupts could stop the tick and keep the CPU for ever, and the whole protection layer would be useless;
- **Latency**: while interrupts are masked, every interrupt waits, even an urgent one that has nothing to do with the shared data. The section must be very short;
- **One CPU** only: on a chip with two cores, masking on core 0 does nothing to core 1, which keeps running and can enter the same section at the same time.

So masking is a good tool inside the kernel, for a few instructions. Tasks need something else: we can't give them access to interrupt masking as is.

9.1.2 Test-and-set flags

A shared flag in memory is useless as it suffers from the exact problem we're trying to solve. We can do with some hardware help. The fix needs the read and the write of the flag to be one atomic step: an atomic read-modify-write. The classic one is **test-and-set**: in one step, read the old value of a word and write 1 into it.

If test-and-set returns 0, the word was free and you set it: you hold the lock. If it returns 1, someone else holds it, and your write of 1 changed nothing. Two tasks can no longer both see 0: the first one's read and write happen together. Many CPUs have such an instruction (x86 has `xchg`, which swaps a register and a word). The Cortex-M3 has none: it does it in another way.

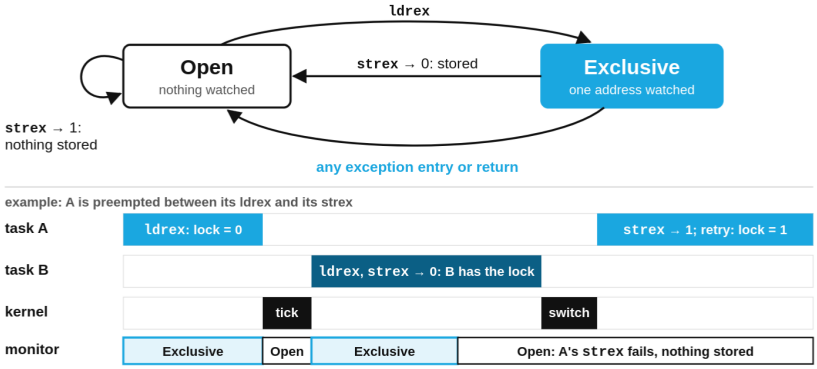
The Cortex-M3 splits the atomic step into two instructions that work together:

```
1 ldrex r1, [r0]      @ r1 = *r0, and start watching: "exclusive access
    begins"
2 strex r2, r1, [r0] @ if nothing happened since ldrex: *r0 = r1, r2 = 0
3                     @ else: store nothing, r2 = 1 (failed)
```

- `ldrex` (*load exclusive*) is a normal load that also says "I want to update this word";
- `strex` (*store exclusive*) stores only if nothing could have touched the word since the ldrex ; its first register tells you if it did (0) or not (1).

If `strex` fails, you simply start again from the `ldrex`- Between the two you can compute anything: test the old value, add 1, ... So the pair gives test-and-set, an atomic increment, and more.

The exclusiveness is ensured by a small piece of hardware: the **exclusive monitor**.



Two states, *Open* and *Exclusive*: `ldrex` sets Exclusive; `strex` stores only in Exclusive, then sets Open. Any exception entry or return sets it to Open (tick, UART interrupt, PendSV, system call), even without a switch. On one CPU another task runs only after an exception: a preempted `strex` fails and A retries.

Another thing to note: to be fast, a CPU and its memory system may let memory accesses complete in a different order than in the program (on bigger cores than ours, very often; the compiler may reorder them too). For a lock that would be fatal: a write of the critical section could become visible after the unlock store, when another task already holds the lock.

`dmb` (*data memory barrier*) allows use to fence this: every memory access before it completes before any memory access after it. The rule is to fence on memory after taking the lock (the section's accesses cannot move before it) and before releasing it. (they cannot

move after it). On one Cortex-M3 the barrier costs almost nothing and rarely changes anything, but with it the lock stays correct on every ARM core.

What we implemented is basically a **spinlock**: while the lock is taken, the task stays in its loop and tries again and again. Waiting by looping on the CPU is busy waiting: it can be done by unprivileged tasks, but it's also really wasteful.

If we have the right primitives available, it's better to do **spin-then-yield**. That is, if the lock is taken, yield the task: the holder runs, frees the lock, and you try again at your next turn.

9.1.3 Semaphores

A **semaphore** S is an integer counter, plus a list of tasks blocked on it. It has two operations, both atomic:

- **Wait**: if S's count is above zero, subtract one from it. Otherwise block on the blocked list;
- **Signal**: if the blocked list holds blocked tasks, free the oldest one. Otherwise add one from S's count.

A **mutex** (from *mutual exclusion*) is a lock with an owner: the task that locked it is the only one allowed to unlock it. Two operations:

- **Lock**: if free, the caller becomes the owner; if taken, the caller waits (usually blocked, like a semaphore);
- **Unlock**: allowed only to the owner; the next waiting task becomes the owner. An unlock by another task is an error the kernel can report.

If you think about it hard enough, kernel-side a mutex is just a semaphore initialized at one and where only who moves past the wait is allowed to signal. User-side we need stronger semantics.

10 Lesson 05-11-26

10.0.1 Introduction to FreeRTOS

FreeRTOS is a real-time operating system kernel for embedded devices that has been ported to 40 microcontroller platforms. It is distributed under the MIT License.

It is designed to be small and simple. It is mostly written in the C programming language to make it easy to port and maintain. It also comprises a few assembly language functions where needed, mostly in architecture-specific context-switching routines used by the scheduler. These routines, which are CPU-dependent, are called *ports*.

FreeRTOS doesn't contain any device drivers: these are to be provided by board developers in the form of a **BSP** (*Board Support Package*).